

SmartLibrary – Bibliotecă online

1. Introducere și Cerințe Funcționale

SmartLibrary este o aplicație web modernă concepută pentru a eficientiza și digitaliza activitățile dintr-o bibliotecă. Scopul principal al platformei este de a simplifica interacțiunea dintre cititori și bibliotecă prin oferirea unui sistem intuitiv de căutare în timp real și rezervare online a cărților. În plus, aplicația oferă un set de instrumente de administrare pentru bibliotecari și administratori pentru a menține stocurile de cărți, a monitoriza starea împrumuturilor și a gestiona utilizatorii înregistrați.

Aplicația se adresează următoarelor categorii distincte de utilizatori:

- a) **Vizitatori (Utilizatori neînregistrați):** Pot consulta liber catalogul, căuta cărți și vizualiza detalii despre disponibilitate.
- b) **Cititori (Utilizatori înregistrați):** Pot rezerva online cărțile dorite (cu ridicare fizică în maxim 48 de ore), pot anula rezervări active și pot vizualiza propriul istoric de împrumuturi și termenele limită de returnare.
- c) **Bibliotecari (Staff-ul bibliotecii):** Administrează catalogul de cărți (adăugare, modificare, ștergere) și procesează fluxul fizic al cărților (confirmare ridicare rezervare/împrumut și confirmare returnare carte).
- d) **Administratori (Admin):** Au acces deplin la controlul securității sistemului, gestionarea conturilor de utilizatori, crearea de conturi pentru bibliotecari și resetarea parolelor.

Sistemul implementează integral cerințele descrise în documentul inițial prin următoarele cazuri de utilizare (Use Cases - UC):

A. Gestionarea contului

- **UC1 – Înregistrare utilizator:**
 - User Story: Ca utilizator neînregistrat, vreau să mă înregistrez pentru a deveni membru al aplicației.
 - Implementare: Ruta /register (GET/POST) validează introducerea numelui, email-ului unic și a parolei confirmate. Parola este criptată securizat înainte de salvarea în baza de date.

- **UC2 – Autentificare:**

- *User Story:* Ca cititor, vreau să mă autentific pentru a accesa contul meu și funcțiile aplicației.
- *Implementare:* Ruta /login verifică existența email-ului și corectitudinea parolei prin hashing securizat. Sesiunea utilizatorului este inițializată cu datele specifice rolului său (cititor, bibliotecar sau admin).

- **UC3 – Delogare:**

- *User Story:* Ca cititor, vreau să mă deloghez pentru a-mi proteja contul.
- *Implementare:* Ruta /logout curăță complet datele din obiectul session de tip Flask și redirecționează utilizatorul la pagina principală.

B. Căutare și vizualizare

- **UC4 – Căutare carte:**

- *User Story:* Ca utilizator, vreau să caut cărți după titlu, autor sau categorie pentru a găsi rapid o carte dorită.
- *Implementare:* Pagina principală / conține o interfață interactivă de căutare. Client-side-ul folosește JavaScript (Vanilla JS) pentru filtrare instantanee, în timp ce tastezi, după titlu, autor sau categorie, împreună cu o paginare automată a rezultatelor fără a reîncărca pagina.

- **UC5 – Vizualizare detalii carte:**

- *User Story:* Ca utilizator, vreau să văd detaliile unei cărți pentru a verifica disponibilitatea acesteia.
- *Implementare:* Ruta /book/<int:book_id> încarcă din baza de date detaliile complete ale unei cărți (titlu, autor, descriere, copertă, număr total de copii și copii disponibile pentru rezervare).

C. Rezervare și Împrumut

- **UC6 – Rezervare carte:**

- *User Story:* Ca cititor, vreau să rezerv o carte pentru a o putea împrumuta.

- *Implementare:* Ruta /reserve/<int:book_id> permite utilizatorului de tip *cititor* să plaseze o rezervare temporară. Aceasta scade stocul disponibil cu 1 unitate și creează o înregistrare cu statusul pending (termen de ridicare de 48 de ore).
- **UC7 – Anulare rezervare:**
 - *User Story:* Ca cititor, vreau să anulez o rezervare pentru a elibera cartea dacă nu o mai doresc.
 - *Implementare:* Ruta /cancel-reservation/<int:res_id> permite cititorului (sau bibliotecarului) să anuleze o rezervare aflată în starea pending. Cartea este returnată automat în stocul disponibil (available_copies += 1).
- **UC8 – Vizualizare rezervări:**
 - *User Story:* Ca cititor, vreau să văd rezervările mele pentru a urmări activitatea mea.
 - *Implementare:* Ruta /dashboard afișează pentru cititori un istoric detaliat, ordonat cronologic, al rezervărilor proprii, colorate intuitiv după status (În așteptare, Împrumutat, Returnat, Anulat).

D. Administrare

- **UC9 – Adăugare carte:**
 - *User Story:* Ca bibliotecar/admin, vreau să adăugă cărți noi în sistem pentru a actualiza colecția.
 - *Implementare:* Ruta /admin/book/add permite încărcarea fișierelor de imagini (copertă) stocate securizat local și salvarea metadatelor cărții.
- **UC10 – Modificare carte:**
 - *User Story:* Ca bibliotecar/admin, vreau să modific informațiile despre o carte pentru a corecta datele.
 - *Implementare:* Ruta /admin/book/edit/<int:book_id> permite modificarea stocurilor totale și recalcularea dinamică a stocului curent.

- **UC11 – Ștergere carte:**

- *User Story:* Ca bibliotecar/admin, vreau să șterg o carte din sistem pentru a elimina înregistrările inutile.
- *Implementare:* Ruta /admin/book/delete/<int:book_id> șterge înregistrarea din baza de date (cu protecție la nivel SQL împotriva ștergerii cărților cu împrumuturi active prin constrângeri de integritate).

2. Uneltele Utilizate (Tehnologii de dezvoltare)

La baza implementării arhitecturii aplicației SmartLibrary a stat o stivă tehnologică simplă și eficientă, organizată pe trei părți: backend-ul în Python/Flask, baza de date relațională SQLite și interfața realizată cu Bootstrap și JavaScript. Această structură a fost aleasă pentru ca programul să funcționeze fără erori și să fie simplu de instalat și rulat.

2.1 Backend-ul Aplicației

- a) **Python 3.10+:** Limbajul de programare de bază ales pentru flexibilitate, sintaxă curată și suport vast de biblioteci.
- b) **Flask:** Micro-framework web ales datorită simplității și modularității sale. Acesta permite definirea rapidă a rutelor, gestionarea cererilor HTTP, managementul sesiunilor și controlul structurii MVC.
- c) **Jinja2:** Motorul de template-uri integrat în Flask, utilizat pentru generarea dinamică a fișierelor HTML. Permite utilizarea structurilor de control (bucle for, condiții if), moștenirea layout-urilor (*base.html*) și implementarea de filtre personalizate (ex: formatarea datelor calendaristice în format românesc prin filtrul `format_date`).
- d) **Werkzeug Security:** Modulul din cadrul Flask utilizat pentru stocarea securizată a parolilor prin algoritmi de hashing criptografic (pbkdf2:sha256). Parolele utilizatorilor nu sunt niciodată salvate în format text clar în baza de date.

2.2 Baza de Date

- a) **SQLite 3:** Un motor de baze de date relaționale, serverless și lightweight. Fiind încorporat direct ca fișier local (`library.db`), acesta elimină dependențele externe complexe și oferă în același timp suport complet pentru tranzacții ACID.

2.3 Frontend (Interfața Utilizator)

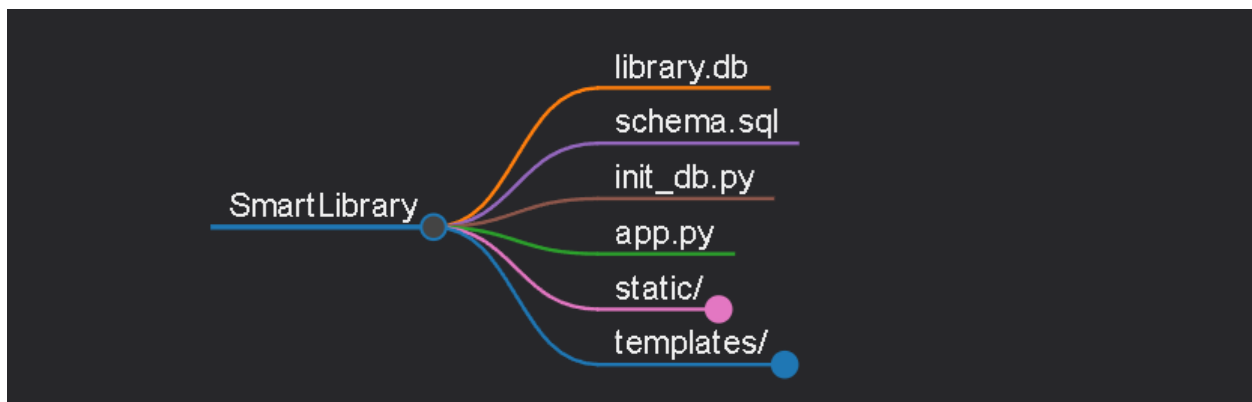
- a) **HTML5 și CSS3:** Responsabile de scheletul paginilor și de designul grafic al aplicației.
- b) **Bootstrap 5 (via CDN):** Framework CSS utilizat pentru crearea unui design modern, adaptiv (responsive) pe orice dispozitiv (telefoane, tablete, laptopuri). A oferit componente predefinite precum carduri de cărți, bare de navigație, ferestre modale interactive și sisteme de grile flexibile.
- c) **Bootstrap Icons:** Pachetul de pictograme vectoriale utilizate pentru o interfață intuitivă și modernă (pictograme pentru butoane, mesaje de avertizare și navigație).
- d) **Vanilla JavaScript (ES6+):** Utilizat direct în frontend pentru:
 - Filtrarea în timp real a cărților din catalog (pe măsură ce utilizatorul tastează titlul/autorul sau alege categoria).
 - Paginarea dinamică la nivelul clientului (afișarea a câte 6 cărți pe pagină pentru a nu supraîncărca interfața).
 - Controlul ferestrelor modale (ex: adăugare carte, editare carte sau confirmarea perioadei de împrumut).

3. Descrierea Arhitecturii Proiectului

Aplicația folosește o arhitectură de tip **Model-View-Controller (MVC)** adaptată pentru structurile specifice Flask. Aceasta permite separarea clară între baza de date, logica de business din backend și elementele vizuale din frontend.

3.1 Organizarea Fișierelor în Proiect

Proiectul este structurat logic în directoare dedicate sarcinilor lor:



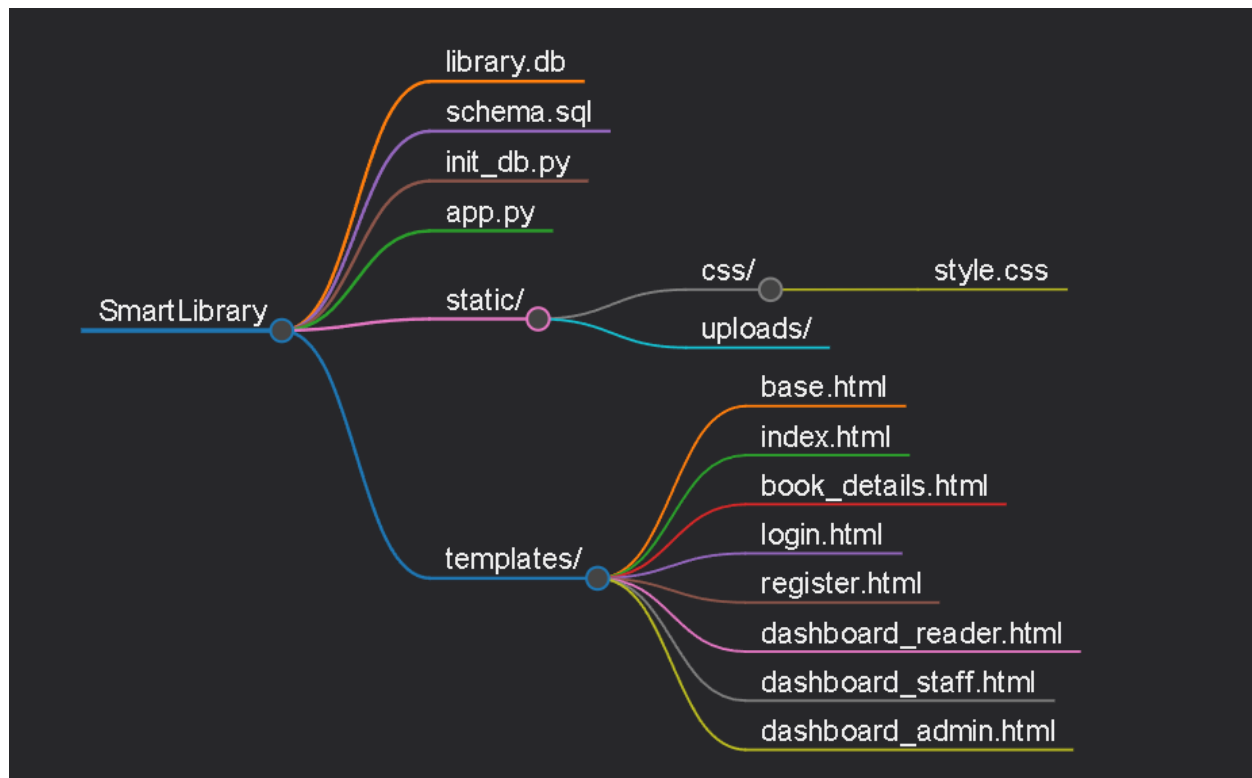


Figura 1. Diagrama organizare fișiere

3.2 Rolul și Funcționalitatea Fiecărui Fișier

a) **app.py (Controller-ul Principal):** Acesta reprezintă inima aplicației. Conține logica de business, rutele URL și interacțiunea cu baza de date. Principalele sale responsabilități sunt:

- **Inițializarea aplicației Flask** și configurarea variabilelor (folderul de upload, cheia secretă a sesiunii).
- **Definirea conexiunilor la baza de date** cu sqlite3.Row pentru acces facil la coloane și activarea constrângerilor SQL pentru chei externe (PRAGMA foreign_keys = ON;).
- **Fluxul automat de curățare** (clean_expired_reservations): Rulat automat la fiecare request de pagină pentru a identifica rezervările în starea pending care au depășit termenul de 48 de ore. Acesta le anulează automat (status = 'cancelled') și returnează cărțile în stoc (available_copies += 1), garantând acuratețea disponibilității.
- **Injectarea contextului de utilizator** (inject_user): Un context processor Jinja2 care face datele utilizatorului logat disponibile în

mod automat în toate șabloanele HTML, facilitând afișarea numelui și verificarea rolurilor.

- **Rutele de Autentificare & Securitate** (/register, /login, /logout).
- **Rutele de Rezervare** (/reserve/<id>, /cancel-reservation/<id>).
- **Rutele Administrative** (/admin/...): Asigură securitatea la nivel de rută, blocând accesul utilizatorilor neautorizați prin verificarea rolului din sesiune.

b) schema.sql (Modelul Date): Definește structura tabelelor relaționale din SQLite. Acesta folosește trei tabele principale (users, books, reservations) cu constrângeri de integritate precum chei unice, valori implicite, verificări (CHECK pe roluri și statusuri) și legături prin chei externe cu declanșare în cascadă la ștergere (ON DELETE CASCADE).

c) init_db.py (Utilitar Inițializare): Un script de tip linie de comandă utilizat pentru configurarea inițială sau resetarea sistemului. Acesta:

- Rulează fișierul schema.sql pentru a crea structura tabelelor.
- Inserează conturi predefinite de test (Admin, Bibliotecari și Cititori cu parole deja criptate) și un set inițial de cărți celebre cu descrieri, categorii și coperti pentru ca aplicația să fie funcțională imediat.

d) Fișierele din templates/ (View-urile): Interfața vizuală folosește sistemul de moștenire modulară. base.html conține scheletul comun HTML, zona de navigare cu meniu adaptiv, secțiunea de mesaje de tip flash (mesaje de succes, eroare, atenționare) și footer-ul. Toate celelalte șabloane moștenesc acest layout folosind blocul {% block content %}, asigurând uniformitate vizuală și evitând duplicarea codului.

e) static/css/style.css (Stilizarea Personalizată): Complementează framework-ul Bootstrap 5 cu elemente estetice specifice unei platforme premium:

- Culori personalizate (tonuri calde de albastru marin, griuri rafinate și accente de chihlimbar).
- Efecte vizuale la trecerea mouse-ului (hover animations) pe cardurile cărților pentru o interfață dinamică.
- Margini rotunjite elegante, umbre fine pentru elementele card și design dedicat insinelor (badges) de disponibilitate.

4. Diagrama Entitate Relație (ERD)

Baza de date a aplicației **SmartLibrary** este complet relațională, formată din 3 entități principale conectate prin relații de integritate referențială de tip 1-to-N.

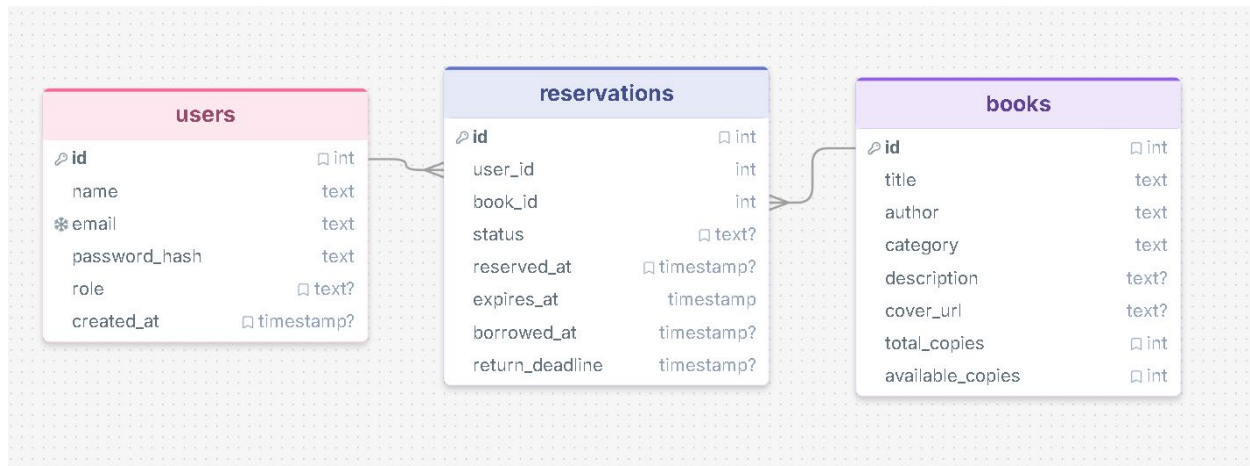


Figura 2. Diagrama entitate relație

Detalierea Relațiilor și Constrângerilor

a) Relația dintre users și reservations (1:N)

- Un utilizator înregistrat (Cititor) poate deține mai multe rezervări de-a lungul timpului (0 sau N rezervări).
- Fiecare înregistrare de rezervare aparține în mod unic unui singur utilizator. Legătura este realizată prin cheia externă `user_id` din tabelul `reservations` care face referință la cheia primară `id` din tabelul `users`.
- Integritate: S-a aplicat clauza `ON DELETE CASCADE`. Dacă un cont de utilizator este șters de către administrator, toate rezervările și împrumuturile asociate contului respectiv se vor șterge automat din baza de date pentru a respecta conformitatea GDPR.

b) Relația dintre books și reservations (1:N)

- O carte din catalog poate face obiectul mai multor rezervări sau împrumuturi succesive efectuate de cititori diferiți (0 sau N rezervări).
- O rezervare specifică face referire la o singură carte din catalog. Legătura este realizată prin cheia externă `book_id` din tabelul `reservations` care face referință la cheia primară `id` din tabelul `books`.

- Integritate: S-a aplicat clauza ON DELETE CASCADE. Dacă o carte este eliminată definitiv din catalog de către un bibliotecar, toate înregistrările sale de rezervări se șterg în cascadă.

c) Mecanismul de disponibilitate a stocului

- Tabelul books stochează independent total_copies (stocul fizic total deținut de bibliotecă) și available_copies (copiile fizice aflate pe raft, disponibile pentru a fi rezervate).
- La realizarea unei rezervări cu succes, stocul de available_copies scade cu 1.
- Dacă rezervarea este anulată de cititor, expiră automat după 48h, sau dacă cartea este returnată fizic la raft de către bibliotecar, stocul de available_copies crește automat cu 1 unitate în mod securizat prin tranzacții SQL (conn.commit()).

5. Descrierea unor cazuri de utilizare ale aplicației

a) Rezervarea unei cărți de către Cititor (UC6 + UC8)

Acest flux permite unui cititor înregistrat și autentificat să rezerve online un exemplar dintr-o carte disponibilă în catalog. Cazul implică o verificare tranzacțională a stocului (`available_copies > 0`), scăderea automată a acestuia cu 1 unitate în caz de succes, prevenirea dublelor rezervări active ale aceluiași utilizator pentru aceeași carte și calcularea termenului limită de ridicare fizică (fixată la exact 48 de ore). De asemenea, presupune curățarea automată în fundal a altor rezervări expirate în momentul accesării paginii.

Pasul 1: Căutarea și selectarea cărții în catalog

- **Ce trebuie făcut:** Utilizatorul (cititorul) accesează pagina principală (/), filtrează cărțile în timp real folosind casetele de căutare din interfață și identifică o carte disponibilă.
- **Efectul în sistem:** Sistemul execută interogarea bazei de date, verifică disponibilitatea fiecărei cărți și afișează badge-ul verde „X disponibile”. Butonul albastru „Rezervă” este activat doar dacă `available_copies > 0`.

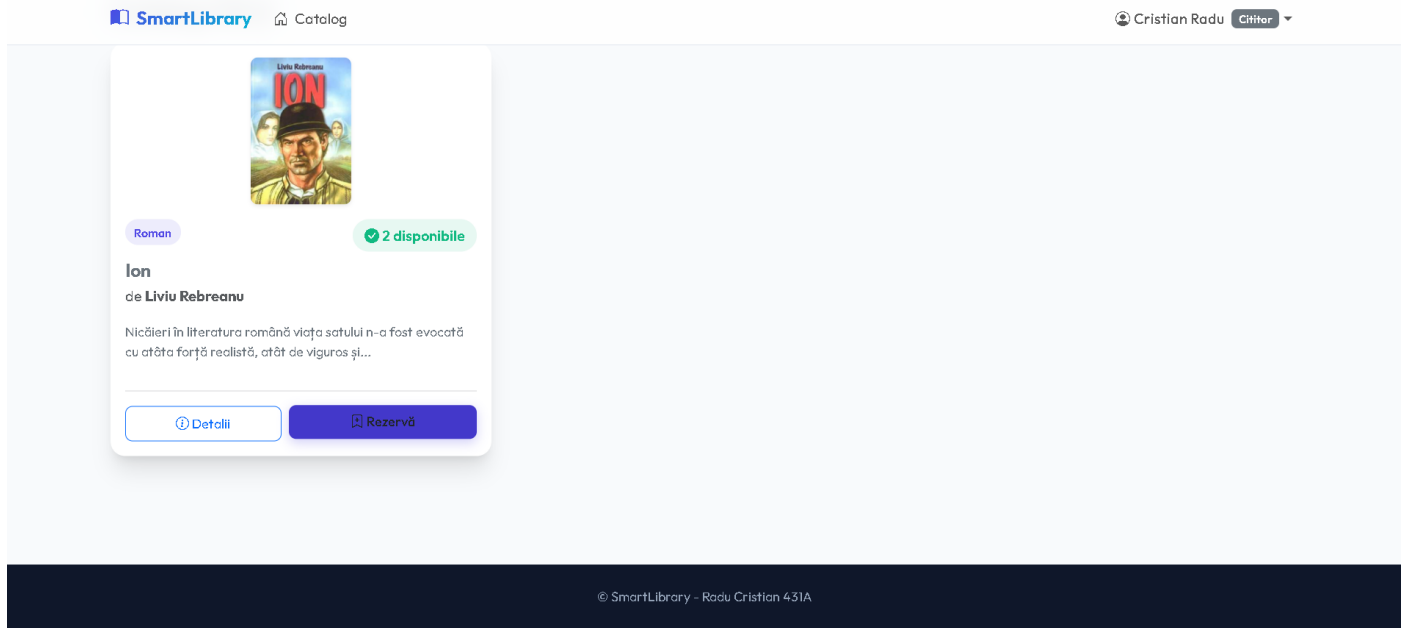


Figura 3. Rezervare carte

Pasul 2: Lansarea rezervării

- **Ce trebuie făcut:** Cititorul dă click pe butonul „Rezervă”.
- **Efectul în sistem:** Serverul primește cererea POST pe ruta `/reserve/<int:book_id>`. Acesta rulează o interogare SQLite pentru a se asigura că utilizatorul nu are deja o rezervare activă. Inițializează o tranzacție: decrementează `available_copies`, înregistrează o linie nouă în tabelul `reservations`, data curentă în `reserved_at` și data curentă + 48 ore în `expires_at`. La final, redirecționează utilizatorul spre dashboard cu un mesaj flash de confirmare.

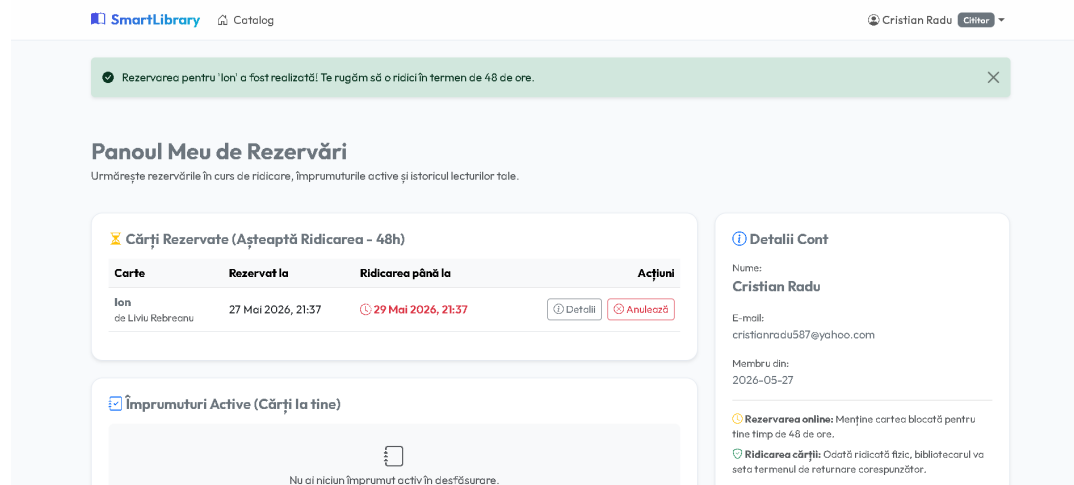


Figura 4. Confirmare rezervare

Pasul 3: Vizualizarea înregistrării în panoul de control personal

- **Ce trebuie făcut:** Utilizatorul verifică secțiunea „Rezervările mele active” în Dashboard-ul său.
- **Efectul în sistem:** Sistemul interoghează tabelul reservations printr-un JOIN cu tabelul books pentru utilizatorul curent și afișează istoricul în dashboard.

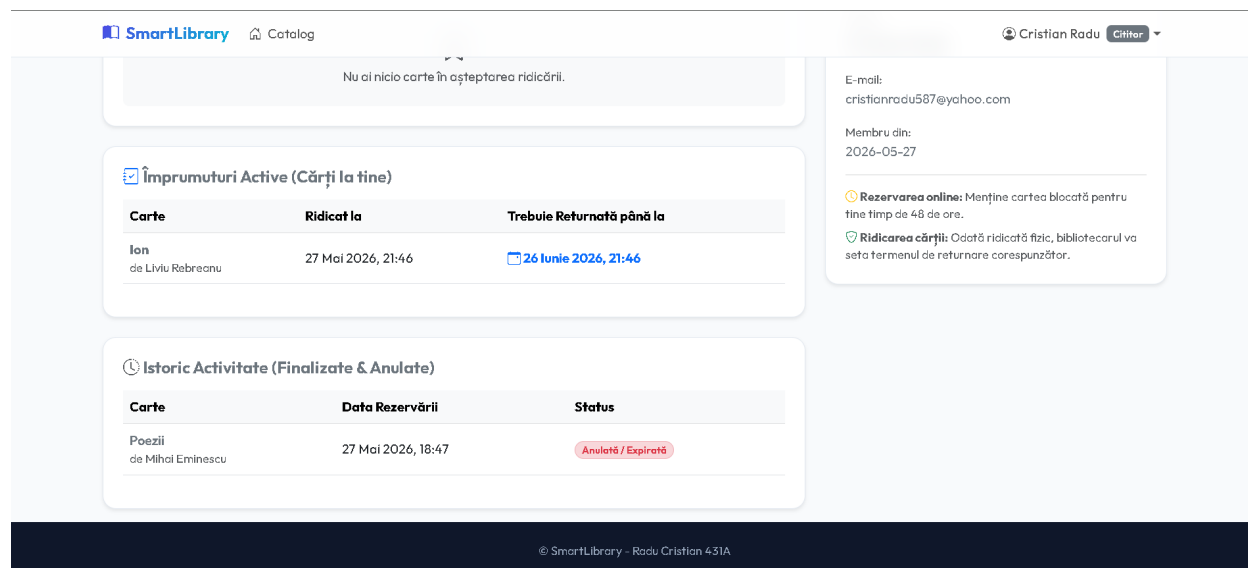


Figura 5. Vizualizare rezervare

b) Gestiunea împrumuturilor de către Bibliotecar (Confirmare Ridicare și Returnare)

Acest flux acoperă interacțiunea fizică dintre cititor și bibliotecar la sediul bibliotecii. Cazul combină confirmarea ridicării fizice a cărții rezervate (care transformă rezervarea în împrumut activ, blocând posibilitatea de anulare a cititorului și setând o dată de returnare configurabilă, implicit 14 zile) cu procesul de returnare ulterioară a cărții la raft (care încheie împrumutul și reintroduce copia fizică în stocul disponibil).

Pasul 1: Identificarea rezervării în panoul bibliotecarului

- **Ce trebuie făcut:** Bibliotecarul se autentifică și accesează secțiunea „Gestiune Rezervări și Împrumuturi” din Panoul de control (/dashboard).
- **Efectul în sistem:** Serverul preia toate rezervările din baza de date printr-o interogare SQL ce reunește tabelele reservations, books și users, oferind detalii despre cine a rezervat și ce carte.

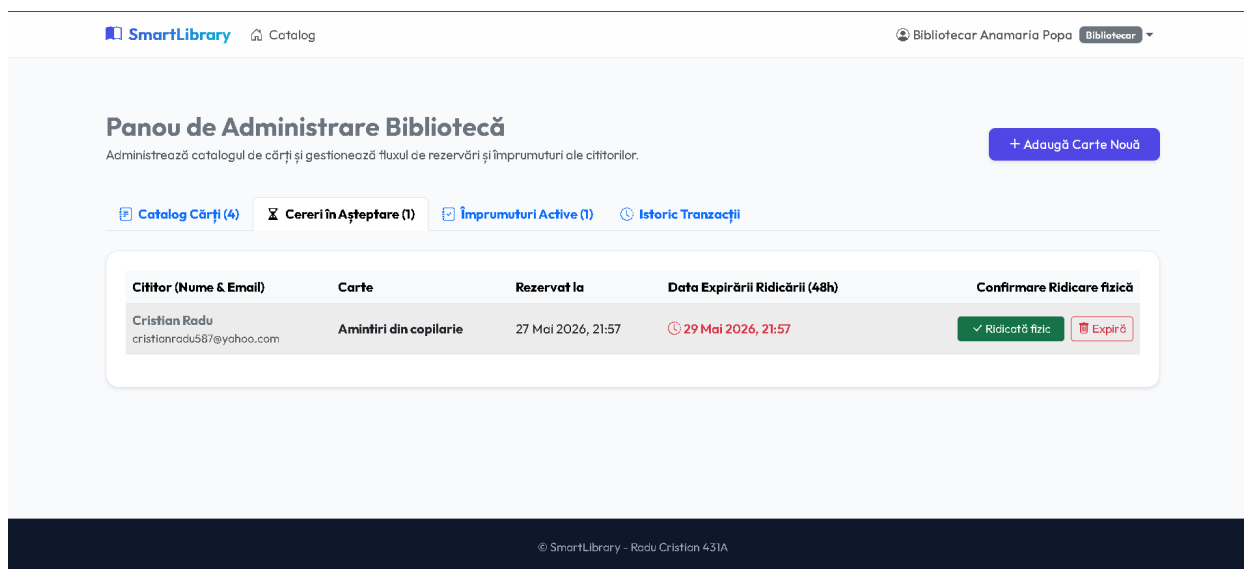


Figura 6. Panou cu cereri în așteptare

Pasul 2: Selectarea termenului de împrumut și confirmarea ridicării

- **Ce trebuie făcut:** Bibliotecarul apasă pe „Ridicată fizic”. Se deschide o fereastră modală unde introduce perioada de împrumut dorită (implicit 14 zile) și confirmă acțiunea.
- **Efectul în sistem:** Serverul primește cererea POST pe ruta `/admin/reservation/pickup/<int:res_id>`. Actualizează în baza de date statusul din pending în borrowed, salvează data curentă în `borrowed_at` și calculează noul termen în `return_deadline` adunând numărul de zile specificat de bibliotecar

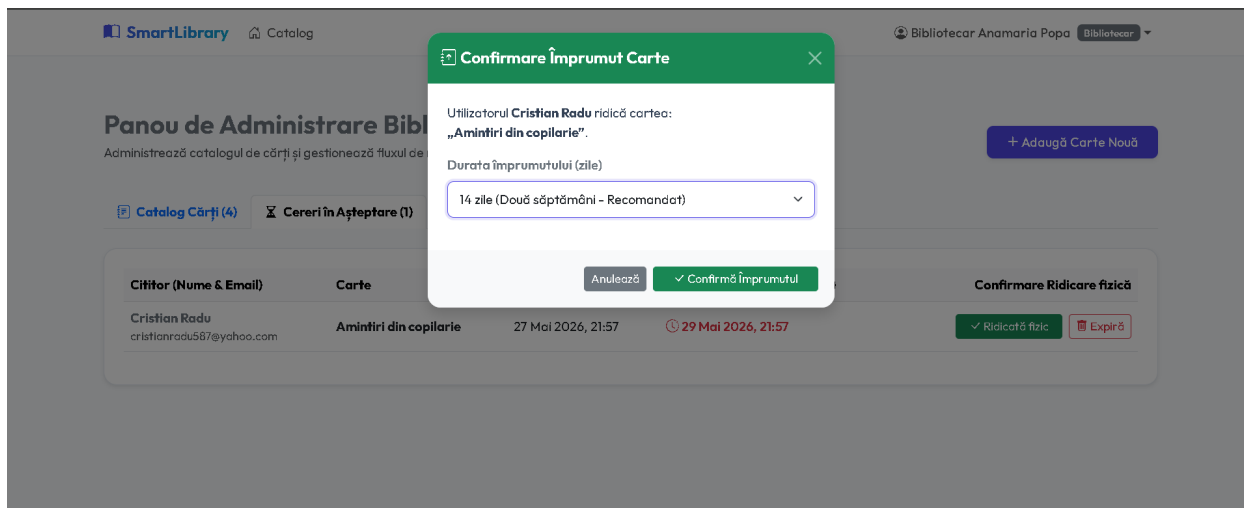


Figura 7. Confirmare împrumut carte

Pasul 3: Vizualizarea împrumutului activ și returnarea cărții

- **Ce trebuie făcut:** După reîncărcarea automată, bibliotecarul verifică starea înregistrării și, la returnarea fizică a cărții la bibliotecă, apasă pe butonul verde „Returnată la raft”.
- **Efectul în sistem:**
 - i. La confirmarea împrumutului: statusul este actualizat la „**Împrumutat (Activ)**” (căsuța verde), data returnării este vizibilă, iar butonul de acțiune devine verde: „**Returnată la raft**”.
 - ii. La confirmarea returnării (ruta /admin/reservation/return/<int:res_id>): statusul devine returned, iar stocul cărții este mărit automat în baza de date ($available_copies = available_copies + 1$).

SmartLibrary Catalog

Bibliotecar Anamaria Popa

Ridicarea cărții a fost confirmată! Împrumutul este acum activ.

Panou de Administrare Bibliotecă

Administrează catalogul de cărți și gestionează fluxul de rezervări și împrumuturi ale cititorilor.

+ Adaugă Carte Nouă

Catalog Cărți (4) Cereri în Așteptare (0) **Împrumuturi Active (2)** Istoric Tranzacții

Cititor	Carte	Data Ridicării	Termen Limită Returnare	Acțiuni
Cristian Radu cristianradu587@yahoo.com	Amințiri din copilărie	27 Mai 2026, 22:03	10 Iunie 2026, 22:03	Returnată la raft
Cristian Radu cristianradu587@yahoo.com	Ion	27 Mai 2026, 21:46	26 Iunie 2026, 21:46	Returnată la raft

Figura 8. Confirmare returnare carte